

1. 퀵 정렬 (Quick Sort) 🚀

'평균적으로 가장 빠른 정렬 방법'으로, '분할 정복' 기법을 사용합니다.

- **핵심 원리:**
 - **피벗 (Pivot):** 리스트를 두 부분으로 나누는 **기준 값**입니다. 강의에서는 보통 첫 번째 요소를 피벗으로 사용합니다.
 - **분할 (Partition):** 피벗을 기준으로, 피벗보다 작은 값은 왼쪽, 큰 값은 오른쪽으로 재배치합니다. 이때, **low**와 **high** 포인터를 사용하여 요소를 이동시키고 교환합니다.
 - **비균등 분할:** 피벗 값에 따라 부분 리스트의 크기가 달라질 수 있습니다.
 - **재귀 호출:** 분할된 각 부분 리스트에 대해 재귀적으로 퀵 정렬을 반복합니다.
- **구현 과정:**
 1. **첫 번째 피벗 설정:** 보통 리스트의 첫 번째 요소를 피벗으로 설정합니다.
 2. **분할:** 피벗을 기준으로 작은 값과 큰 값을 좌우로 나누어 정렬합니다. 피벗은 제자리를 찾습니다.
 3. **재귀적 분할:** 나뉜 두 부분 리스트에 대해 다시 피벗을 설정하고 분할 과정을 반복합니다.
- **시간 복잡도:**
 - **최선/평균:** $O(N \log N)$ (균등 분할 시).
 - **최악:** $O(N^2)$ (극도로 불균등하게 분할될 때, 예: 이미 정렬된 리스트에서 항상 첫 요소를 피벗으로 선택하는 경우).
- **장점:** 평균적으로 가장 빠르며, 추가적인 메모리 공간이 적게 듭니다.
- **단점:** 최악의 경우 성능이 저하될 수 있으며, **불안정 정렬**입니다 (동일 값의 상대적 위치 보장 안됨).

2. 힙 정렬 (Heap Sort) 🌳

힙 자료구조를 활용한 정렬 알고리즘입니다.

- **핵심 원리:**
 - **힙 (Heap):** 완전 이진 트리 형태의 자료구조로, **최대 힙** (부모 노드가 자식 노드보다 항상 큼) 또는 **최소 힙** (부모 노드가 자식 노드보다 항상 작음)으로 구성됩니다.
 - **정렬 과정:** 전체 요소를 힙으로 구성한 후, 힙에서 한 번에 하나의 요소를 삭제하여 정렬된 배열에 저장하는 방식으로 정렬합니다. 최대 힙에서는 가장 큰 요소(루트)가 먼저 삭제되어 뒤부터 채워지고, 최소 힙에서는 가장 작은 요소가 먼저 삭제되어 앞부터 채워집니다.
 - 힙에서의 삽입/삭제 연산은 $O(\log N)$ 시간이 소요됩니다.
- **시간 복잡도:** $O(N \log N)$ (힙 생성 및 N번의 삭제 연산).
- **특징:**
 - 전체 자료를 모두 정렬할 필요 없이, 가장 큰 값 몇 개 또는 가장 작은 값 몇 개만 필요할 때 유용합니다.
 - **불안정 정렬**입니다.

3. 기수 정렬 (Radix Sort) [12]

레코드를 비교하지 않고, 자릿수(또는 문자 위치)를 기준으로 버킷(임시 공간)을 활용하는 정렬 방법입니다.

- **핵심 원리:**
 - **버킷 (Bucket):** 임시 저장 공간으로, 각 자릿수(0~9) 또는 문자(A~Z)에 해당하는 버킷을 만듭니다. (버킷은 큐(Queue)로 구현되는 경우가 많음)

- **낮은 자릿수부터 정렬:** 가장 낮은 자릿수(예: 1의 자리)부터 시작하여 각 숫자를 해당 버킷에 넣고 순서대로 다시 꺼냅니다.
- **반복:** 다음 자릿수(예: 10의 자리, 100의 자리)에 대해 이 과정을 반복합니다.
- **동작 방식 예시 (두 자리 숫자 정렬):**
 1. **1의 자리 기준 정렬:** 모든 숫자를 1의 자리에 따라 0~9 버킷에 넣고 순서대로 꺼냅니다.
 2. **10의 자리 기준 정렬:** 1의 자리 정렬된 결과물을 10의 자리에 따라 0~9 버킷에 넣고 순서대로 꺼냅니다.
 3. **최종 결과:** 10의 자리까지 정렬된 최종 리스트를 얻습니다.
- **시간 복잡도:** $O(DN)$ (D 는 키의 자릿수, N 은 레코드 수).
 - 키의 자릿수 D 가 작을수록(예: 10 이하) 매우 빠릅니다.
- **특징:**
 - **비교 연산을 사용하지 않습니다.**
 - **안정적 정렬**입니다.
 - **제약 조건:** 정렬할 레코드 타입이 **한정적**입니다 (숫자 또는 문자로 구성되고 동일한 길이를 가져야 함). 실수, 한글, 한자 등에는 적용하기 어렵습니다.
 - 버킷의 개수는 키 표현 방법과 밀접하게 관련됩니다 (예: 10진수 10개, 알파벳 26개, 2진수 2개).

교수님께서 중요하다고 강조하신 부분 (시험 출제 예상)

교수님은 다음 내용을 핵심적으로 강조하셨으며, 특히 **프로그래밍 실습**의 중요성을 여러 번 언급했습니다.

- **퀵 정렬의 피벗 개념과 분할 과정:** 피벗을 기준으로 작은 값/큰 값으로 나누는 원리. 최악의 경우 ($O(N^2)$) 발생 조건 (불균등 분할, 이미 정렬된 리스트).
- **힙 정렬의 시간 복잡도 ($O(N \log N)$) 및 활용성:** 전체 정렬보다 '상위 몇 개' 또는 '하위 몇 개'를 찾을 때 유용하다는 점.
- **기수 정렬의 버킷 개념과 정렬 방식:** 비교 연산을 사용하지 않는다는 특징, 낮은 자릿수부터 높은 자릿수 순으로 정렬하는 과정.
- **기수 정렬의 제약 사항:** 숫자 또는 문자로 구성된 동일 길이의 키에만 적용 가능 (실수, 한글, 한자 등은 어렵다).
- **모든 정렬 알고리즘의 프로그래밍 실습:**
 - 교재의 함수를 활용하되, **main** 함수를 직접 수정하여 난수 대신 고정된 요소를 입력.
 - **단계별 정렬 과정**을 **printf** 등을 사용하여 **출력**하고 결과를 확인하는 것이 필수. (특히 퀵 정렬, 기수 정렬의 단계별 출력 강조)